



ESPE

UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

TEMA: Arquitectura

Departamento: Departamento de Ciencias de la Computación

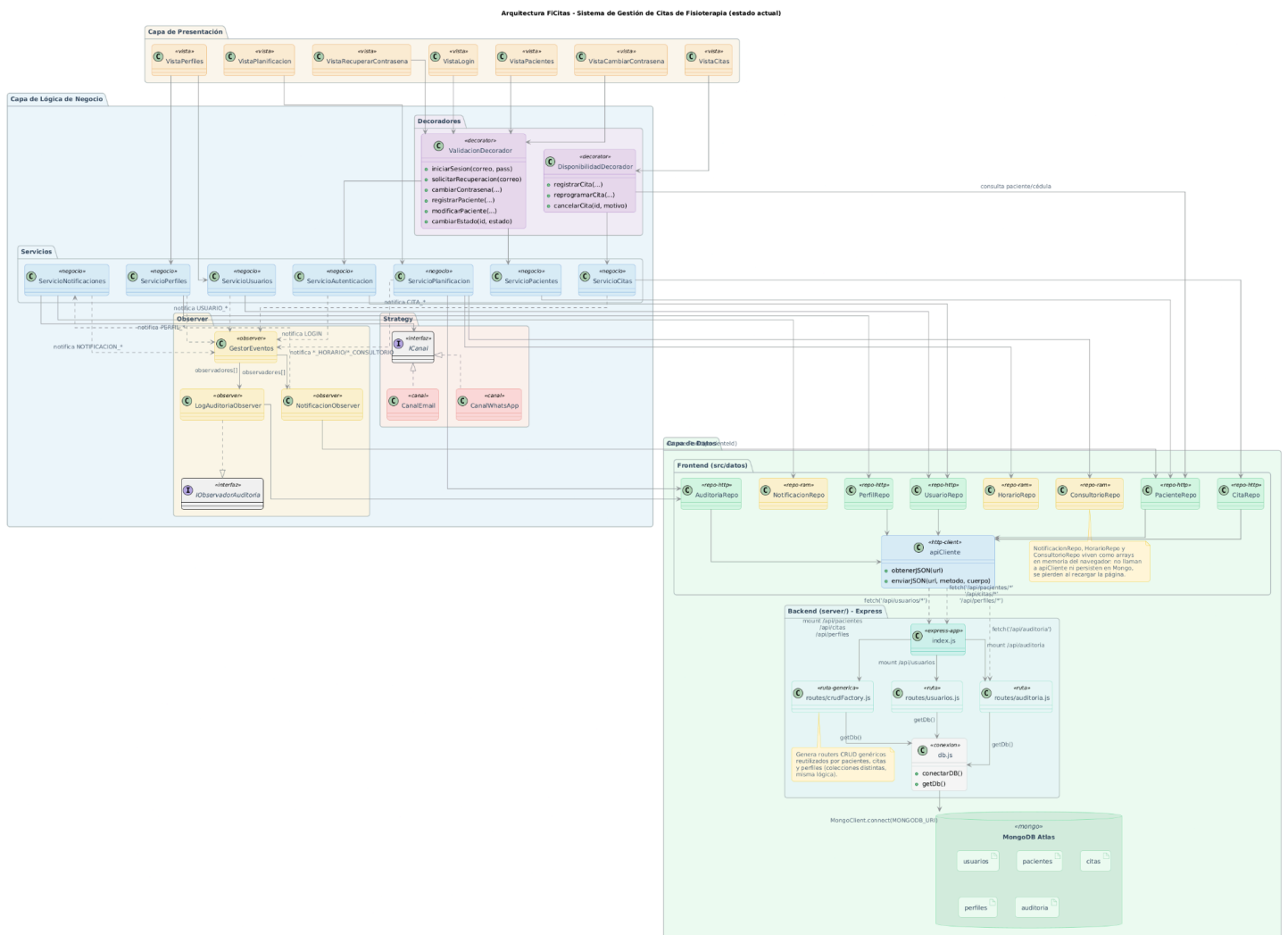
Carrera: Ingeniería de Software
Asignatura: Análisis y Diseño de Software
NRC: 30723

Integrantes Grupo 3:

- Ayuquina Danny.
- Cañarte Saray.
- Villagómez Doménica.

PROYECTO: FICITAS

1. Diagrama de Arquitectura 3 capas



2. Justificación

FiCitas se organiza en 3 capas con bajo acoplamiento: la Presentación solo conoce la interfaz de los servicios/decoradores que instancia, la Lógica de Negocio no sabe cómo ni dónde se persisten los datos, y la persistencia real ocurre en un backend Express separado que habla con MongoDB Atlas — el frontend nunca toca Mongo directamente, todo pasa por `fetch()` a `/api/*`.

Capa de Presentación

Vista	Requisito	Qué captura/hace	Servicio(s) que usa
VistaLogin	REQ000	Correo y contraseña; muestra errores de validación	ValidacionDecorador → ServicioAutenticacion
VistaRecuperarContraseña	REQ001	Correo para solicitar código de recuperación	ValidacionDecorador → ServicioAutenticacion
VistaCambiarContraseña	REQ001	Código de verificación + nueva contraseña	ValidacionDecorador → ServicioAutenticacion
VistaCitas	REQ003	Creación, reprogramación, cancelación y consulta de citas	DisponibilidadDecorador → ServicioCitas
VistaPacientes	REQ004	Registro, modificación, cambio de estado y consulta de pacientes	ValidacionDecorador → ServicioPacientes
VistaPerfiles	REQ005	Alta/edición de perfiles y de cuentas de usuario	ServicioPerfiles + ServicioUsuarios (sin decorador)
VistaPlanificacion	REQ006	Horarios, consultorios y asignación de fisioterapeutas	ServicioPlanificacion (sin decorador)

No existe una VistaNotificaciones (REQ002): las notificaciones se muestran como avisos in-app disparados por NotificacionObserver tras eventos de citas, no como una pantalla CRUD independiente.

Capa de Lógica de Negocio

Elemento	Tipo	Qué hace
ValidacionDecorador	Decorator	Envuelve ServicioAutenticacion y ServicioPacientes; valida formato, inyección de script y reglas de negocio antes de delegar
DisponibilidadDecorador	Decorator	Envuelve ServicioCitas; valida duración máxima, fecha futura, existencia/estado del paciente y solapamiento de horario/consultorio
ServicioAutenticacion	Servicio	Login, recuperación y cambio de contraseña vía UsuarioRepo
ServicioCitas	Servicio	CRUD de citas; publica eventos CITA_REGISTRADA/CANCELADA/REPROGRAMADA
ServicioPacientes	Servicio	CRUD de pacientes y cambio de estado
ServicioPerfiles	Servicio	CRUD de perfiles; impide desactivar el único perfil Administrador activo

ServicioUsuarios	Servicio	Alta/edición de cuentas de usuario ligadas a un perfil
ServicioPlanificacion	Servicio	CRUD de horarios/consultorios; detecta solapamientos; registra su propia auditoría
ServicioNotificaciones	Servicio	Envía la notificación por el canal correspondiente (Strategy) o la deja pendiente
GestorEventos	Observer (subject)	Mantiene la lista de observadores y los notifica tras cada operación relevante
IObservadorAuditoria	Interfaz Observer	Contrato actualizar(evento)
LogAuditoriaObserver	Observer concreto	Persiste todo evento publicado como registro de auditoría
NotificacionObserver	Observer concreto	Filtra eventos CITA_* y delega el envío a ServicioNotificaciones
ICanal / CanalEmail / CanalWhatsApp	Strategy	Seleccionan y ejecutan el canal de envío de la notificación

Capa de Datos

Frontend (src/datos/)

Repositorio	Respaldo	Operaciones clave
UsuarioRepo	HTTP → /api/usuarios	listar, guardar, login, recuperación, cambio de contraseña
PacienteRepo	HTTP → /api/pacientes	listar, guardar, actualizar, obtenerPorId
CitaRepo	HTTP → /api/citas	listar, guardar, actualizar, obtenerPorId
PerfilRepo	HTTP → /api/perfiles	listar, obtenerPorId, guardar, actualizar
AuditoriaRepo	HTTP → /api/auditoria	listar, guardar
NotificacionRepo	Memoria (array)	listar, guardar, obtenerPendientes — sin persistencia
HorarioRepo	Memoria (array)	listar, guardar, actualizar, eliminar — sin persistencia
ConsultorioRepo	Memoria (array)	listar, guardar, eliminar — sin persistencia

Todos los repos HTTP pasan por apiCliente.js (obtenerJSON/enviarJSON, wrapper de fetch).

Backend (server/)

Componente	Rol
index.js	Bootstrap de Express; monta las rutas /api/* y sirve el frontend como estático
routes/usuarios.js	Ruta manual: hash con bcrypt, login, recuperación de contraseña
routes/crudFactory.js	Genera routers CRUD genéricos, reutilizado por pacientes, citas y perfiles

routes/auditoria.js	Ruta CRUD de auditoría
db.js	Conexión singleton a MongoDB Atlas (conectarDB() / getDb())

MongoDB Atlas

Colección	Identificador de documento
usuarios	id numérico (Date.now())
pacientes	id numérico
citas	id numérico
perfiles	id numérico
auditoria	id numérico

Nota: MongoDB usa _id (ObjectId) por defecto, pero FiCitas ignora _id y filtra/actualiza por su propio campo id numérico generado en el cliente.